Software Measurement & Metrics

Lecture 8

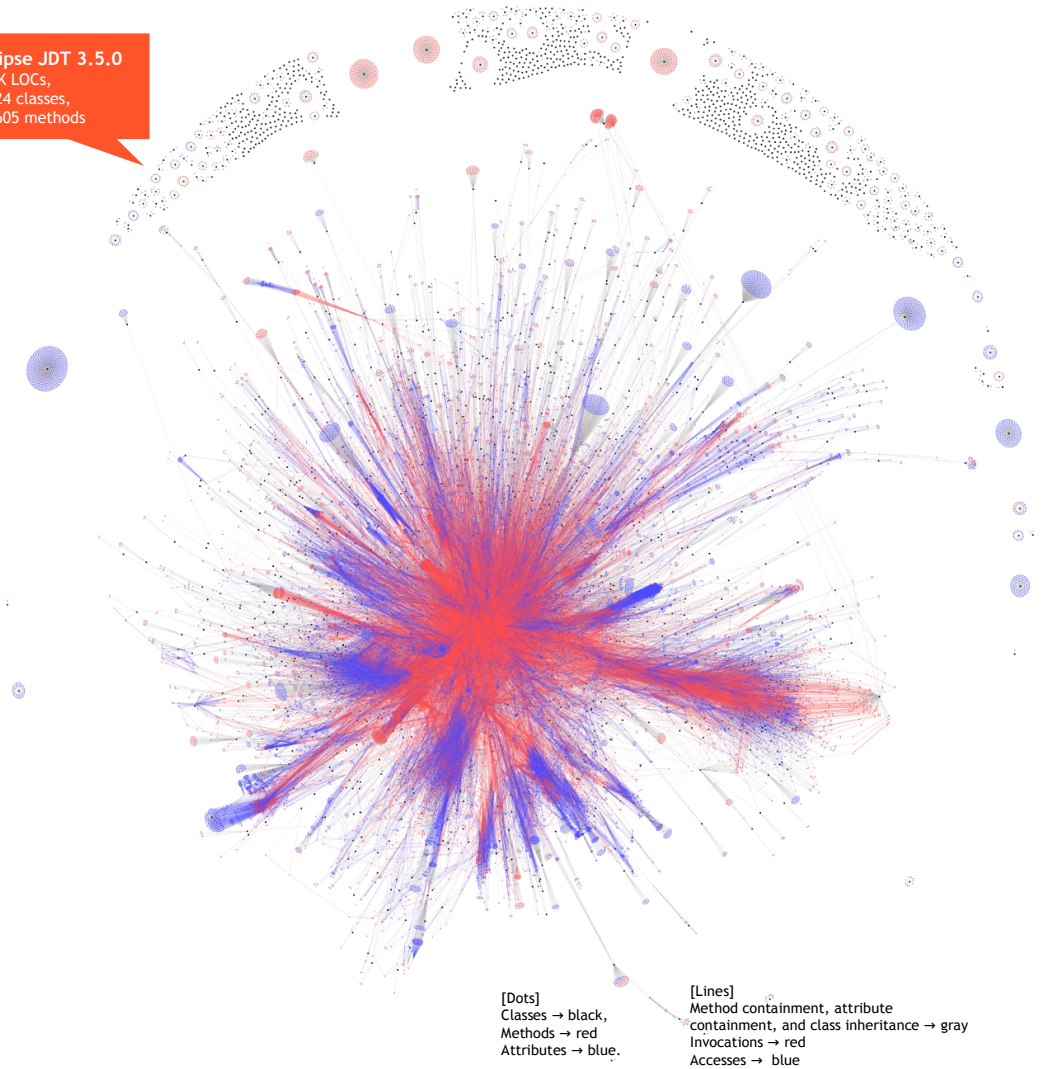
Stanislav Chren

stanislav.chren@aalto.fi

Aalto University
School of Science

Introduction

- Modern systems are very large & complex in terms of structure & runtime behaviour
- We need ways to understand attributes of software, represent it in a concise way and use it to track for software & development process improvement
- Software Measurement and Metrics are one of the aspects we can consider

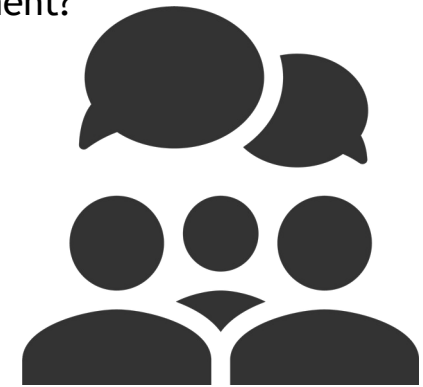


Background on Software Measurement

Measurement

- **Measurement** is the process by which **numbers** or **symbols** are assigned to **attributes of entities** in **the real world** in such a way as to describe them according to **clearly defined rules**
  (Fenton & Pfleeger, 1997)

Why do we need software measurement?



Measurement

- **Measurement** is the process by which **numbers** or **symbols** are assigned to **attributes of entities** in **the real world** in such a way as to describe them according to **clearly defined rules**
  (Fenton & Pfleeger, 1997)

Why do we need software measurement?

To avoid anecdotal evidence without a clear research

To increase the visibility and the understanding of the process

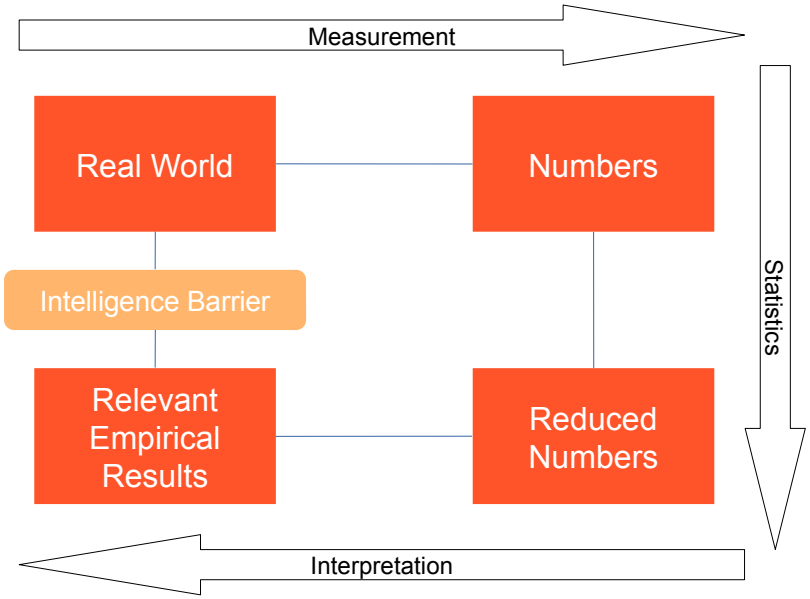
To analyze the software development process

To make predictions through statistical models

To set measurable targets for the software products

Measurement Process

- The measurement process goes from the **real world** to the **numerical representation**
- Interpretation goes from the **numerical representation** to the **relevant empirical results**



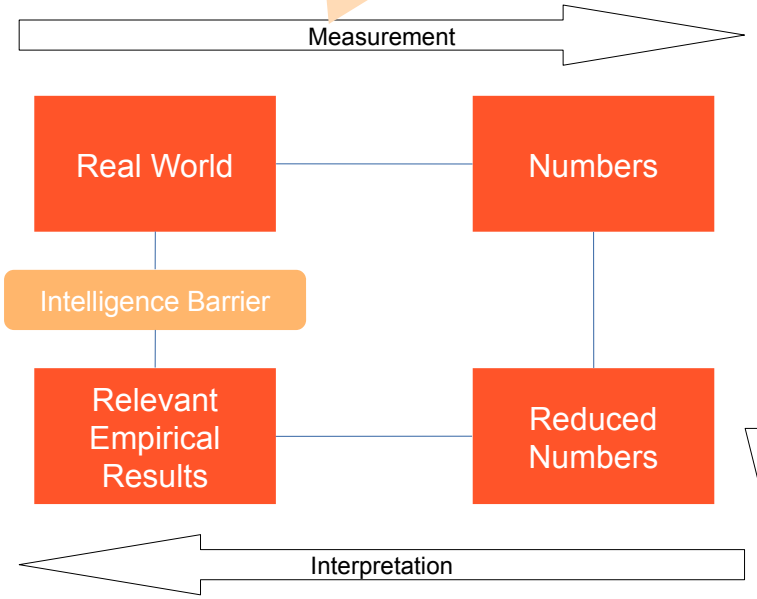
Ford, Gary A. Lecture Notes on Engineering Measurement for Software Engineers. Carnegie Mellon University, Software Engineering Institute, 1993.

Measurement Process

- The measurement process goes from the **real world** to the **numerical representation**
- Interpretation goes from the **numerical representation** to the **relevant empirical results**

A **measure** is a mapping between the **real world** and **mathematical/formal world**

- Different mappings give different views of the world (height, weight, …)
- The **mapping relates attributes to mathematical objects**; **it does not relate entities to mathematical objects**
- **Direct** vs **indirect**
- **Objective** vs **subjective**



Missing intuitive mapping between real world objects and numerical results, i.e. barrier between questions about objects and the answers to those questions.

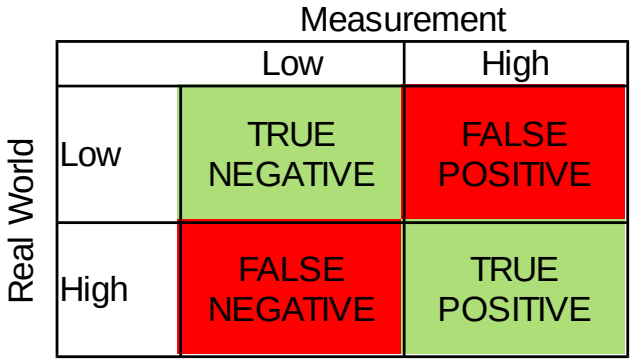
E.g. for aggregation of results (avg, sum, min/max …)

Reasoning about results that provides answer in terms of the original object domain rather than in the domain of numbers.

Ford, Gary A. Lecture Notes on Engineering Measurement for Software Engineers. Carnegie Mellon University, Software Engineering Institute, 1993.

Valid Measures

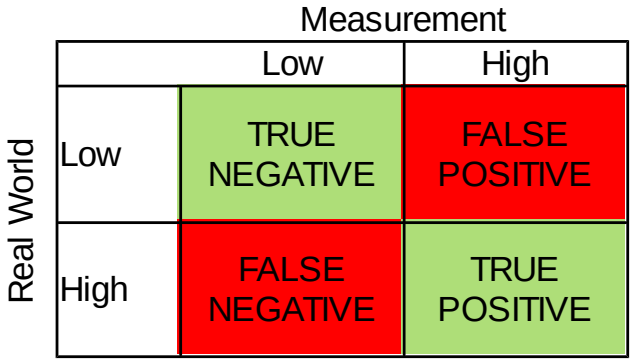
- The validity of a measure **depends on definition of the attribute coherent** with the **specification of the real world**



- *Q: Is LOC a valid measure of productivity?*

Valid Measures

- The validity of a measure **depends on definition of the attribute coherent** with the **specification of the real world**



- **Q: Is LOC a valid measure of productivity?**
  - For example, 100K print() statements vs 100K of complex loops and algorithms
  - Can we compare LOC between different projects?

Valid Measures - Example

- **Code coverage** is a measure giving an indication of how much of the source code has been run (“covered”) by running the tests

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]   private static boolean[] crossedOut;
[07]   private static int[] result;
[08]   public static int[] generatePrimes(int maxValue){
[09]     if (maxValue < 2){
[10]       return new int[0];
[11]     }else{
[12]       uncrossIntegersUpTo(maxValue);
[13]       crossOutMultiples();
[14]       putUncrossedIntegersIntoResult();
[15]       return result;
[16]     }
[17]   }
[18] }
```

Valid Measures - Example

- **Code coverage** is a measure giving an indication of how much of the source code has been run (“covered”) by running the tests

- *“...A program with high code coverage has been **more thoroughly tested** and has a **lower chance of containing software bugs** than a program with low code coverage…”*
(Wikipedia, until 2022)

- **Q: Would you consider code coverage as a valid measure of how much thoroughly one software project has been tested and how good the tests are?**
  - Suppose you have two projects with the code coverage P1: 70% vs P2: 80%
  - Would you generally consider P2 to be “better” (more accurately) tested than P1?

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]   private static boolean[] crossedOut;
[07]   private static int[] result;
[08]   public static int[] generatePrimes(int maxValue){
[09]     if (maxValue < 2){
[10]       return new int[0];
[11]     }else{
[12]       uncrossIntegersUpTo(maxValue);
[13]       crossOutMultiples();
[14]       putUncrossedIntegersIntoResult();
[15]       return result;
[16]     }
[17]   }
[18] }
```

Valid Measures - Example

- **Code coverage** is a measure giving an indication of how much of the source code has been run (“covered”) by running the tests

- *“...A program with high code coverage has been **more thoroughly tested** and has a **lower chance of containing software bugs** than a program with low code coverage…”*
(Wikipedia, until 2022)

- **Q: Would you consider code coverage as a valid measure of how much thoroughly one software project has been tested and how good the tests are?**
  - Suppose you have two projects with the code coverage P1: 70% vs P2: 80%
  - Would you generally consider P2 to be “better” (more accurately) tested than P1?

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]   private static boolean[] crossedOut;
[07]   private static int[] result;
[08]   public static int[] generatePrimes(int maxValue){
[09]     if (maxValue < 2){
[10]       return new int[0];
[11]     }else{
[12]       uncrossIntegersUpTo(maxValue);
[13]       crossOutMultiples();
[14]       putUncrossedIntegersIntoResult();
[15]       return result;
[16]     }
[17]   }
[18] }
```

Q: Would you consider every test covering the same lines as equal?

Valid Measures - Example

- **Code coverage** is a measure giving an indication of how much of the source code has been run ("covered") by running the tests

- *"...A program with high code coverage has been **more thoroughly tested** and has a **lower chance of containing software bugs** than a program with low code coverage..."*
(Wikipedia, until 2022)

- **Q: Would you consider code coverage as a valid measure of how much thoroughly one software project has been tested and how good the tests are?**
  - Suppose you have two projects with the code coverage P1: 70% vs P2: 80%
  - Would you generally consider P2 to be "better" (more accurately) tested than P1?

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]   private static boolean[] crossedOut;
[07]   private static int[] result;
[08]   public static int[] generatePrimes(int maxValue){
[09]     if (maxValue < 2){
[10]       return new int[0];
[11]     }else{
[12]       uncrossIntegersUpTo(maxValue);
[13]       crossOutMultiples();
[14]       putUncrossedIntegersIntoResult();
[15]       return result;
[16]     }
[17]   }
[18] }
```

Q: Would you consider every test covering the same lines as equal?

```
[01] double div (int x, int y){
[02]    return x/y;
[03] }
```

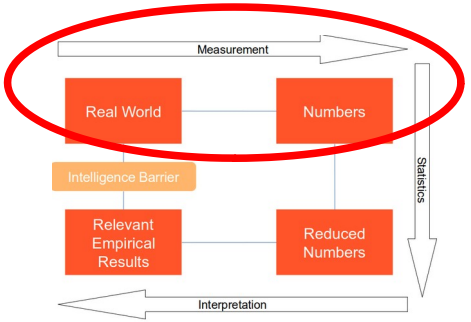
Coverage 100%

```
assertEquals(1.0, div(1,1));
```

Coverage 100%

```
assertEquals(0.66, div(2,3), 0.1);
```

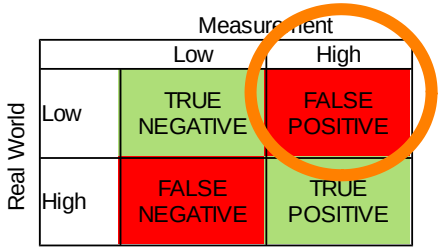
Valid Measures - Example

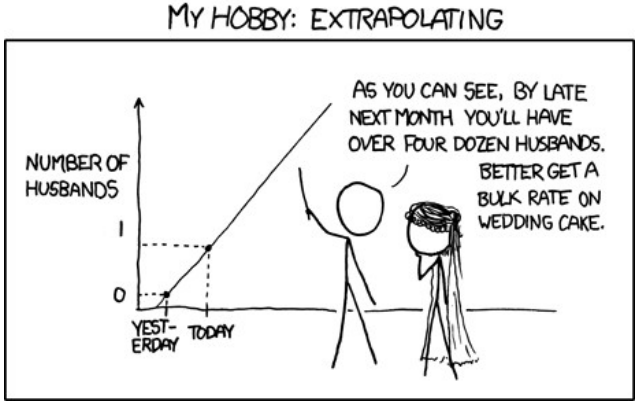


- According to **Martin Fowler**:
- → *"Test coverage is a **useful tool for finding untested parts of a codebase**. Test coverage is **of little use as a numeric statement of how good your tests are**"*

  (https://martinfowler.com/bliki/TestCoverage.html)

- In this case, **we do not respect the representation condition**: when we assign symbols to the attributes of entities we need to preserve the meaning of relationships when moving entities from the real world to the numerical world



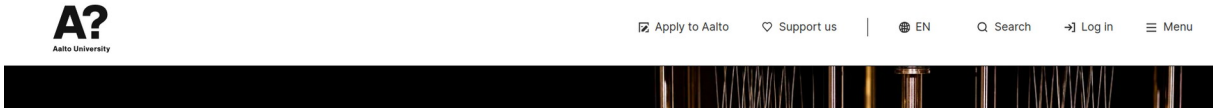


https://xkcd.com/605/

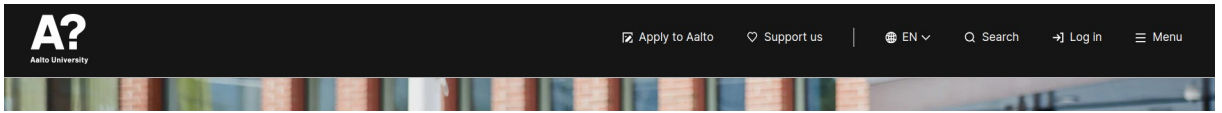
Valid Measures

- A/B Testing is a kind of **randomized experiment** in which you can propose **two variants** of the same application to the users
- Example
  - We set-up an experiment with two browsers and two variations of the same webpage
  - **Conversion Rate:** *% of users completing an action*

Variant A



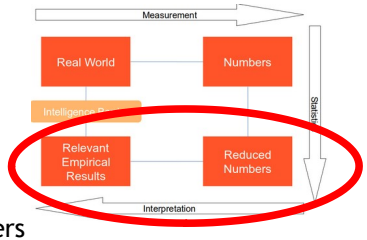
Variant B



200 users	Conv Rate A	Conv Rate B
Firefox	87.50%	100.00%
Chrome	50.00%	62.50%

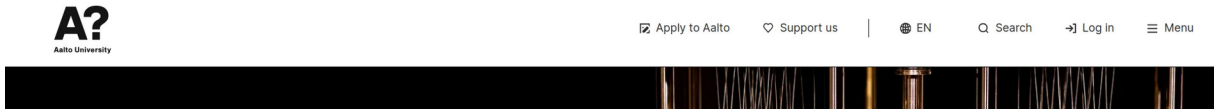
Q: What can you conclude? Which alternative is better?

Valid Measures

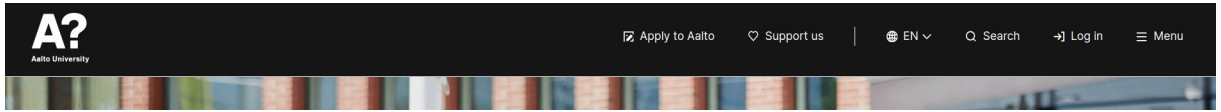


- A/B Testing is a kind of **randomized experiment** in which you can propose **two variants** of the same application to the users
- Example
  - We set-up an experiment with two browsers and two variations of the same webpage
  - **Conversion Rate:** *% of users completing an action*

Variant A



Variant B



200 users	Conv Rate A	Conv Rate B
Firefox	87.50%	100.00%
Chrome	50.00%	62.50%

Q: What can you conclude? Which alternative is better?

	Conv Rate A		Conv Rate B	
Firefox	70/80	= 87.5%	20/20	= 100%
Chrome	10/20	= 50%	50/80	= 62.5%
Both	80/100	= 80%	70/100	= 70%

users for the variant

users converted

See Simpson's paradox:
https://plato.stanford.edu/entries/paradox-simpson/

Measurement Scales

- Every measurement is mapped to a scale (**nominal, ordinal, interval, rational**)
- Considering the scale is important for the allowed operations

Name of prog. Language (e.g. Java, C++,...)

Ranking of failures/severity (e.g. critical, minor,...)

Start date, end date of activities

Lines of code (as a measure of program size)

	≠,=	<,>	min,max	median	avg	prop
Nominal →						
Ordinal →						
Interval →						
Rational →						

Measurement Scales - Example

Level	Severity	Description
6	Blocker	Prevents function from being used, no work-around, blocking progress on multiple fronts
5	Critical	Prevents function from being used, no work-around
4	Major	Prevents function from being used, but a work-around is possible
3	Normal	A problem making a function difficult to use but no special work-around is required
2	Minor	A problem not affecting the actual function, but the behavior is not natural
1	Trivial	A problem not affecting the actual function, a typo would be an example

Order	Severity	P1	P2
6	Blocker	2	10
5	Critical	36	19
4	Major	25	22
3	Normal	15	32
2	Minor	2	5
1	Trivial	121	113

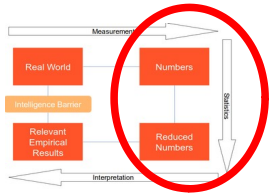
- **Q: Is it meaningful to use the weighted average to compare two projects in terms of severity of the open issues?**

$$Sev(P_n) = avg\left(\sum issues_i * weight_i\right)$$

$$Sev(P_1) = avg(2 * 6 + 36 * 5 + 25 * 4 + 15 * 3 + 2 * 2 + 121 * 1) = 77$$

$$Sev(P_2) = avg(10 * 6 + 19 * 5 + 22 * 4 + 32 * 3 + 5 * 2 + 113 * 1) = 77$$

Measurement Scales - Example



Level	Severity	Description
6	Blocker	Prevents function from being used, no work-around, blocking progress on multiple fronts
5	Critical	Prevents function from being used, no work-around
4	Major	Prevents function from being used, but a work-around is possible
3	Normal	A problem making a function difficult to use but no special work-around is required
2	Minor	A problem not affecting the actual function, but the behavior is not natural
1	Trivial	A problem not affecting the actual function, a typo would be an example

Order	Severity	P1	P2
6	Blocker	2	10
5	Critical	36	19
4	Major	25	22
3	Normal	15	32
2	Minor	2	5
1	Trivial	121	113

- **Q: Is it meaningful to use the weighted average to compare two projects in terms of severity of the open issues?**

$$Sev(P_n) = avg\left(\sum issues_i * weight_i\right)$$

$$Sev(P_1) = avg(2*6+36*5+25*4+15*3+2*2+121*1) = 77$$

$$Sev(P_2) = avg(10*6+19*5+22*4+32*3+5*2+113*1) = 77$$

Software Metrics

Measures of Size

Q: What size measures can you come up with in this example?

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]   private static boolean[] crossedOut;
[07]   private static int[] result;
[08]   public static int[] generatePrimes(int maxValue){
[09]     if (maxValue < 2){
[10]       return new int[0];
[11]     }else{
[12]       uncrossIntegersUpTo(maxValue);
[13]       crossOutMultiples();
[14]       putUncrossedIntegersIntoResult();
[15]       return result;
[16]     }
[17]   }
[18] }
```

Measures of Size

LOC = 18
(Lines of Code)

CLOC = 3
(Commented
Lines of Code)

NCLOC = 3
(Commented
Lines of Code)

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]   private static boolean[] crossedOut;
[07]   private static int[] result;
[08]   public static int[] generatePrimes(int maxValue){
[09]     if (maxValue < 2){
[10]       return new int[0];
[11]     }else{
[12]       uncrossIntegersUpTo(maxValue);
[13]       crossOutMultiples();
[14]       putUncrossedIntegersIntoResult();
[15]       return result;
[16]     }
[17]   }
[18] }
```

Measures of Size

LOC = 18
(Lines of Code)

CLOC = 3
(Commented
Lines of Code)

NCLOC = 3
(Commented
Lines of Code)

NOC = 1
(Number of
Classes)

NOM = 1
(Number of
Methods)

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]   private static boolean[] crossedOut;
[07]   private static int[] result;
[08]   public static int[] generatePrimes(int maxValue){
[09]     if (maxValue < 2){
[10]       return new int[0];
[11]     }else{
[12]       uncrossIntegersUpTo(maxValue);
[13]       crossOutMultiples();
[14]       putUncrossedIntegersIntoResult();
[15]       return result;
[16]     }
[17]   }
[18] }
```

Measures of Size

LOC = 18
(Lines of Code)

CLOC = 3
(Commented
Lines of Code)

NCLOC = 3
(Commented
Lines of Code)

NOC = 1
(Number of
Classes)

NOM = 1
(Number of
Methods)

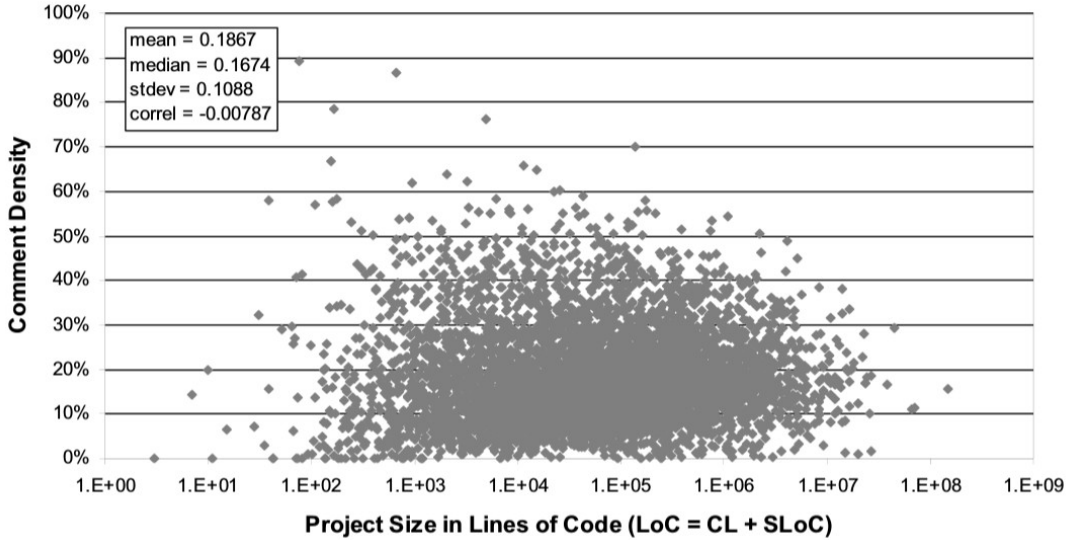
```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]   private static boolean[] crossedOut;
[07]   private static int[] result;
[08]   public static int[] generatePrimes(int maxValue){
[09]     if (maxValue < 2){
[10]       return new int[0];
[11]     }else{
[12]       uncrossIntegersUpTo(maxValue);
[13]       crossOutMultiples();
[14]       putUncrossedIntegersIntoResult();
[15]       return result;
[16]     }
[17]   }
[18] }
```

Q: Can you think of something more useful to report than CLOC?

Measures of Size

- Size can be used for **normalization of existing measures**
  - E.g. instead of CLOC, we can use Comments Density (CD)

$$CD = \frac{CLOCs}{LOCs} = \frac{3}{18} = 0.16$$

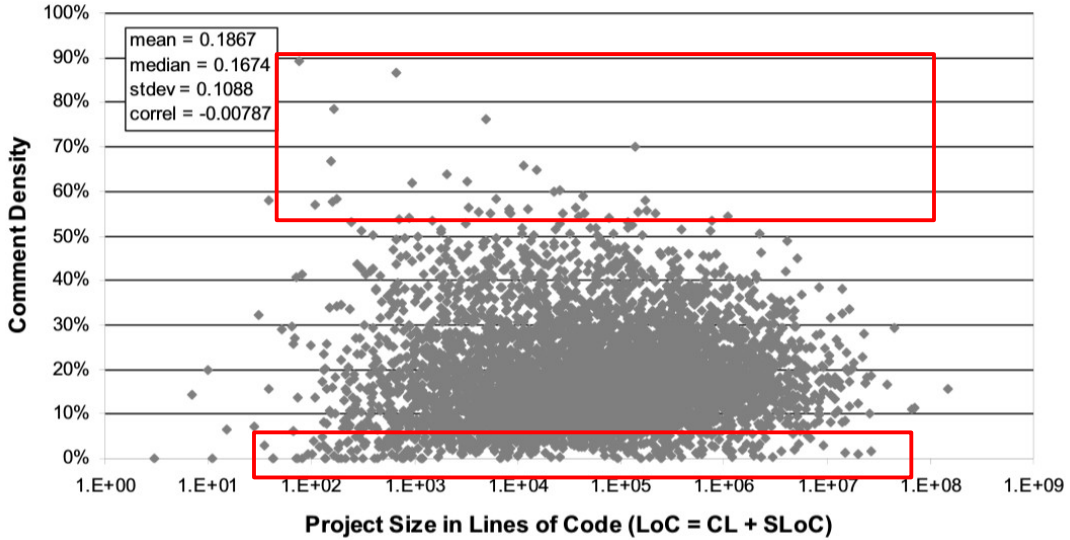


O. Arafat and D. Riehle, "The comment density of open source software code," in 31st International Conference on Software Engineering - Companion Volume, 2009. ICSE-Companion 2009, 2009, pp. 195–198.

Measures of Size

- Size can be used for **normalization of existing measures**
  - E.g. instead of CLOC, we can use Comments Density (CD)

$$CD = \frac{CLOCs}{LOCs} = \frac{3}{18} = 0.16$$



O. Arafat and D. Riehle, "The comment density of open source software code," in 31st International Conference on Software Engineering - Companion Volume, 2009. ICSE-Companion 2009, 2009, pp. 195–198.

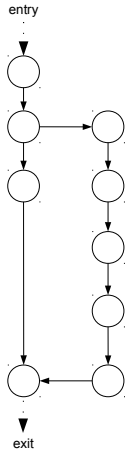
Measures of Complexity

- McCabe's Cyclomatic Complexity (CC)
  - v(G) = |E|-|N|+2P

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]   private static boolean[] crossedOut;
[07]   private static int[] result;
[08]   public static int[] generatePrimes(int maxValue){
[09]     if (maxValue < 2){
[10]       return new int[0];
[11]     }else{
[12]       uncrossIntegersUpTo(maxValue);
[13]       crossOutMultiples();
[14]       putUncrossedIntegersIntoResult();
[15]       return result;
[16]     }
[17]   }
[18] }
```

Measures of Complexity

- McCabe's Cyclomatic Complexity (CC)
  - v(G) = |E|-|N|+2P



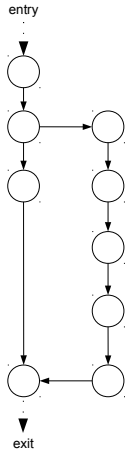
CC = 2

v(G)=|E|-|N|+2
v(G)=9-9+2=2

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]   private static boolean[] crossedOut;
[07]   private static int[] result;
[08]   public static int[] generatePrimes(int maxValue){
[09]     if (maxValue < 2){
[10]       return new int[0];
[11]     }else{
[12]       uncrossIntegersUpTo(maxValue);
[13]       crossOutMultiples();
[14]       putUncrossedIntegersIntoResult();
[15]       return result;
[16]     }
[17]   }
[18] }
```

Measures of Complexity

- McCabe's Cyclomatic Complexity (CC)
  - v(G) = |E|-|N|+2P



CC = 2

v(G)=|E|-|N|+2
v(G)=9-9+2=2

Q: How can be the Cyclomatic Complexity useful?

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]   private static boolean[] crossedOut;
[07]   private static int[] result;
[08]   public static int[] generatePrimes(int maxValue){
[09]     if (maxValue < 2){
[10]       return new int[0];
[11]     }else{
[12]       uncrossIntegersUpTo(maxValue);
[13]       crossOutMultiples();
[14]       putUncrossedIntegersIntoResult();
[15]       return result;
[16]     }
[17]   }
[18] }
```

Measures of Complexity

- CC is more about syntactic complexity, **NOT semantic complexity**

```
[04] public class PrimeGenerator
[05] {
[06]   private static boolean[] crossedOut;
[07]   private static int[] result;
[08]
[09]   public static int[] generatePrimes(int maxValue){
[10]     if (maxValue < 2){
[11]       return new int[0];
[12]     }else{
[13]       uncrossIntegersUpTo(maxValue);
[14]       crossOutMultiples();
[15]       putUncrossedIntegersIntoResult();
[16]       return result;
[17]   }
[18] }
```

```
[04] public class A
[05] {
[06]   private static boolean[] c;
[07]   private static int[] b;
[08]
[09]   public static int[] generate(int m){
[10]     if (m < 2){
[11]       return new int[0];
[12]     }else{
[13]       methodOne(m);
[14]       methodTwo();
[15]       methodThree();
[16]       return b;
[17]   }
[18] }
```

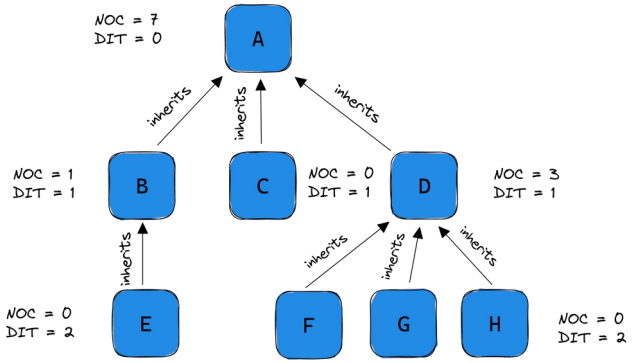
Q: Which code fragment is easier to understand?

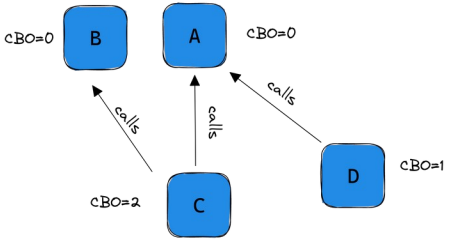
- Be careful when **comparing projects in terms of average complexity**

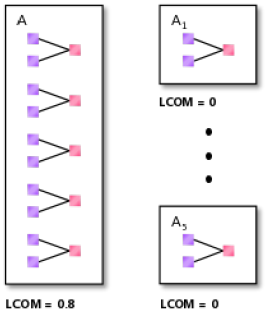
Object-oriented Metrics

Chidamber & Kemerer Suite

- **WMC**: Weighted methods per class
- **DIT**: Depth of Inheritance Tree
- **NOC**: Number of Children
- **CBO**: Coupling between object classes
- **RFC**: Response for a class
- **LCOM**: Lack of cohesion in methods

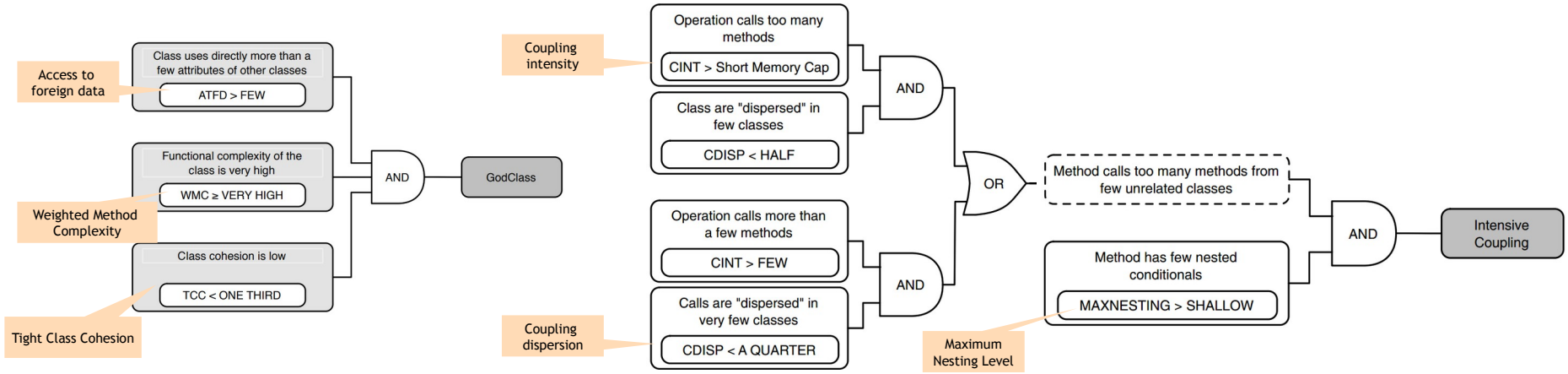






Source: NDepend tool documentation

Object-oriented Metrics – Code Smells



Michele Lanza, and Radu Marinescu. Object-oriented metrics in practice. Springer, 2007.

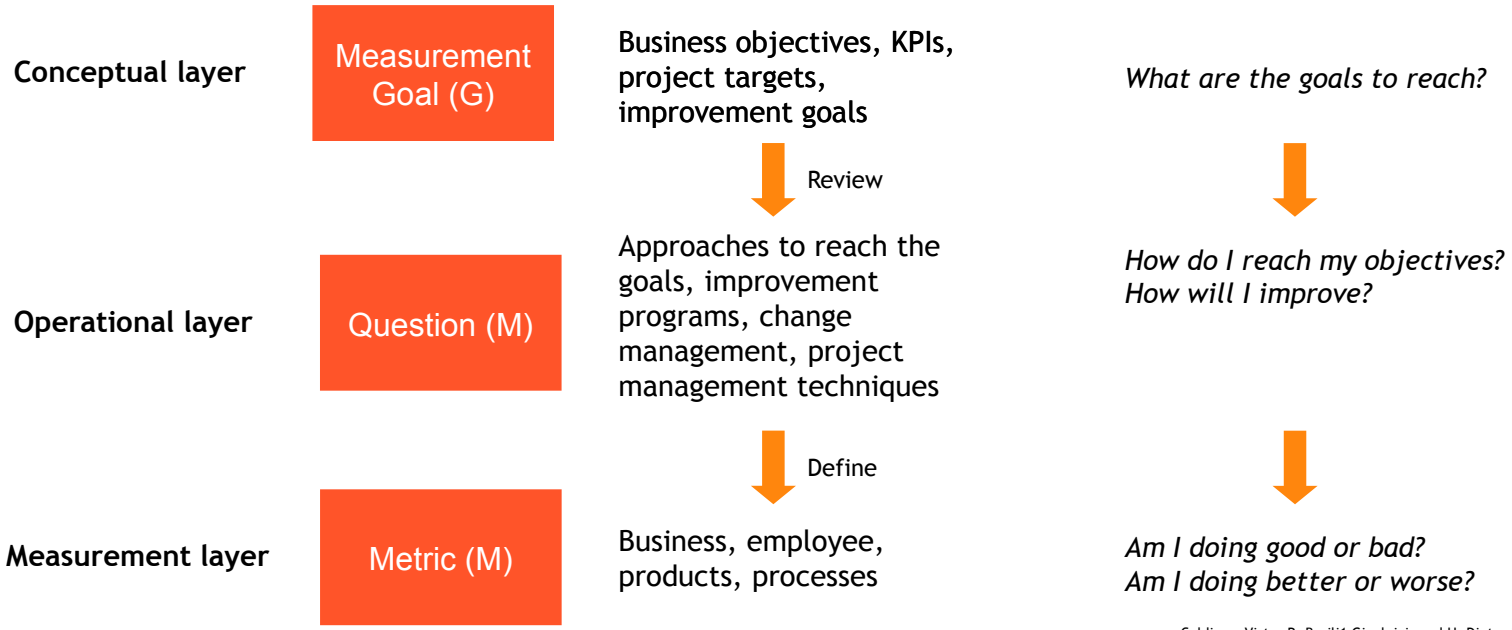
Software Measurement Models

Software Measurement Pitfalls

Collecting meaningless measurements	• Measurement must be goal-driven
Not analysing measurements	• Numbers need detailed analysis
Setting unrealistic targets	• Targets should not be uniquely defined based on the numbers
Paralysis by analysis	• Measurement is a key activity in management, not a separate activity

The Goal-Question-Metric (GQM)

- Introduced in 1986 by Rombach and Basili
- Top-down deductive instrument to derive suitable measure from prescribed goals



Caldiera, Victor R. Basili1 Gianluigi, and H. Dieter Rombach. "The goal question metric approach." Encyclopedia of software engineering (1994): 528-532.

GQM - Examples

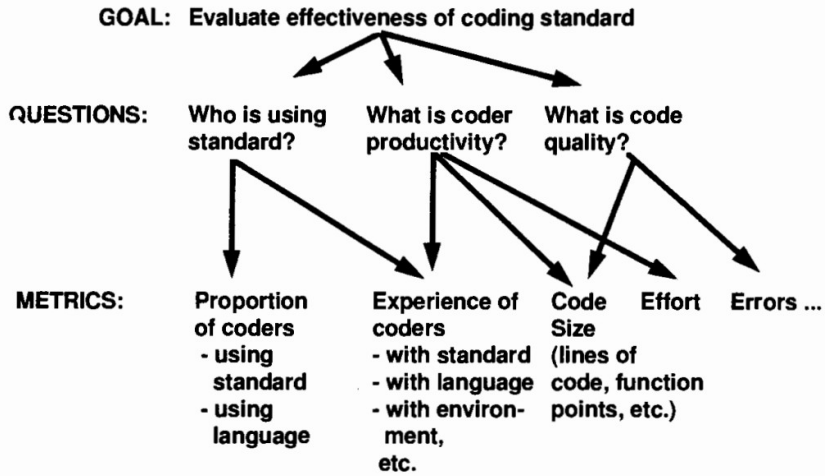


Figure 3.2: Example of deriving metrics from goals and questions

Fenton, Norman, and James Bieman. Software metrics: a rigorous and practical approach. CRC press, 2014.

Aalto University
School of Science

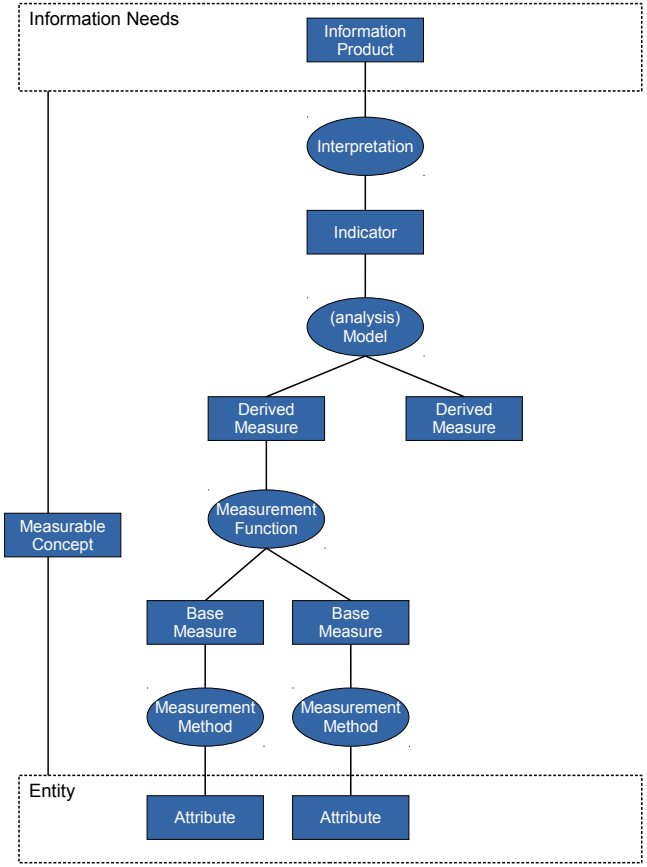
Table 3.3: Examples of AT&T goals, questions and metrics (Barnard and Price, 1994)

Goal	Questions	Metrics
Plan	How much does the inspection process cost?	Average effort per KLOC Percentage of reinspections
	How much calendar time does the inspection process take?	Average effort per KLOC Total KLOC inspected
Monitor and control	What is the quality of the inspected software?	Average faults detected per KLOC Average inspection rate Average preparation rate
	To what degree did the staff conform to the procedures?	Average inspection rate Average preparation rate Average lines of code inspected Percentage of reinspections
	What is the status of the inspection process?	Total KLOC inspected
Improve	How effective is the inspection process?	Defect removal efficiency Average faults detected per KLOC Average inspection rate Average preparation rate Average lines of code inspected
	What is the productivity of the inspection process?	Average effort per fault detected Average inspection rate Average preparation rate Average lines of code inspected

Barnard, Jack, and Art Price. "Managing code inspection information." IEEE Software 11.2 (1994): 59-69.

Measurement Information Model

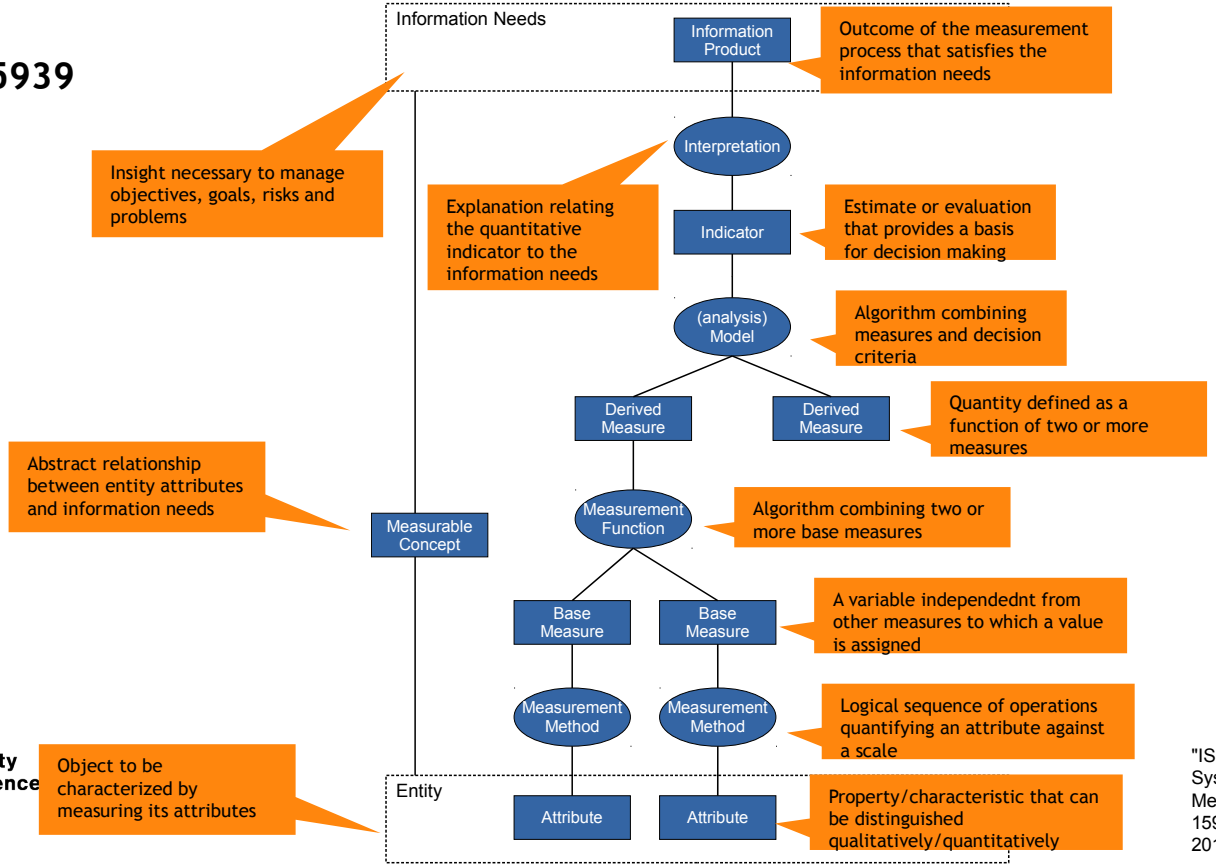
- ISO/IEC 15939



"ISO/IEC/IEEE International Standard - Systems and software engineering--Measurement process," in ISO/IEC/IEEE 15939:2017(E) , vol., no., pp.1-49, 20 April 2017, doi: 10.1109/IEEESTD.2017.7907158.

Measurement Information Model

- ISO/IEC 15939



"ISO/IEC/IEEE International Standard - Systems and software engineering--Measurement process," in ISO/IEC/IEEE 15939:2017(E) , vol., no., pp.1-49, 20 April 2017, doi: 10.1109/IEEESTD.2017.7907158.

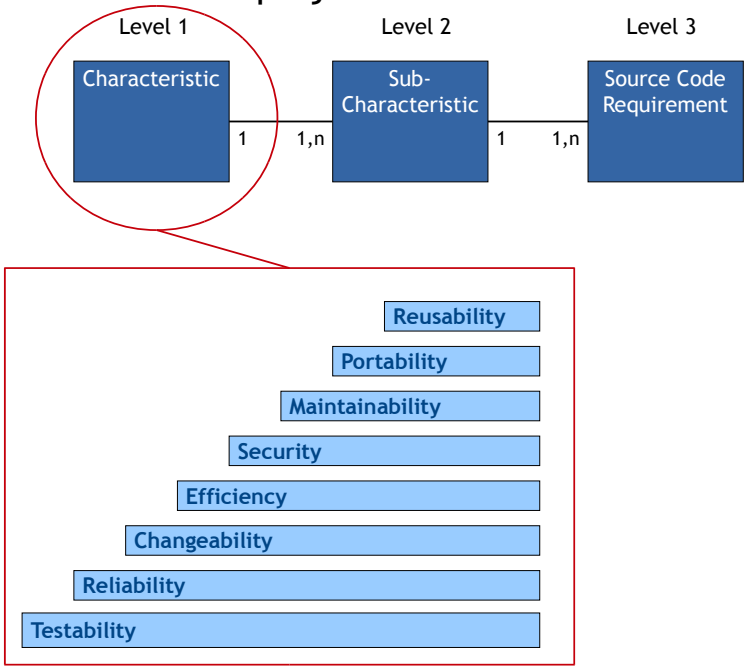
ISO/IEC 15939 - Example

Information Need	Estimate productivity of future project
Measurable Concept	Project productivity
Relevant Entities	1. Requirements implemented by past projects 2. Effort expended by past projects
Attributes	1. Shall Statements 2. Timecard entries (recording effort)
Base Measures	1. Project X Requirements 2. Project X Hours of Effort
Measurement Method	1. Count "Shalls" in Requirements Specification 2. Add timecard entries together for Project X
Type of Measurement Method	1. Objective 2. Objective
Scale	1. Integers from zero to infinity 2. Real numbers from zero to infinity
Type of Scale	1. Ratio 2. Ratio
Unit of Measurement	1. Line 2. Hour
Derived Measure	Project X Productivity
Measurement Function	Divide Project X Requirements Implemented by Project X Hours of Effort
Indicator	Average productivity
Model	Compute mean and standard deviation of all project productivity values.
Decision Criteria	Computed confidence limits based on the standard deviation indicate the likelihood that an actual result close to the average productivity will be achieved. Very wide confidence limits suggest a potentially large departure and the need for contingency planning to deal with this outcome.

"ISO/IEC/IEEE International Standard - Systems and software engineering--Measurement process," in ISO/IEC/IEEE 15939:2017(E) , vol., no., pp.1-49, 20 April 2017, doi: 10.1109/IEEESTD.2017.7907158.

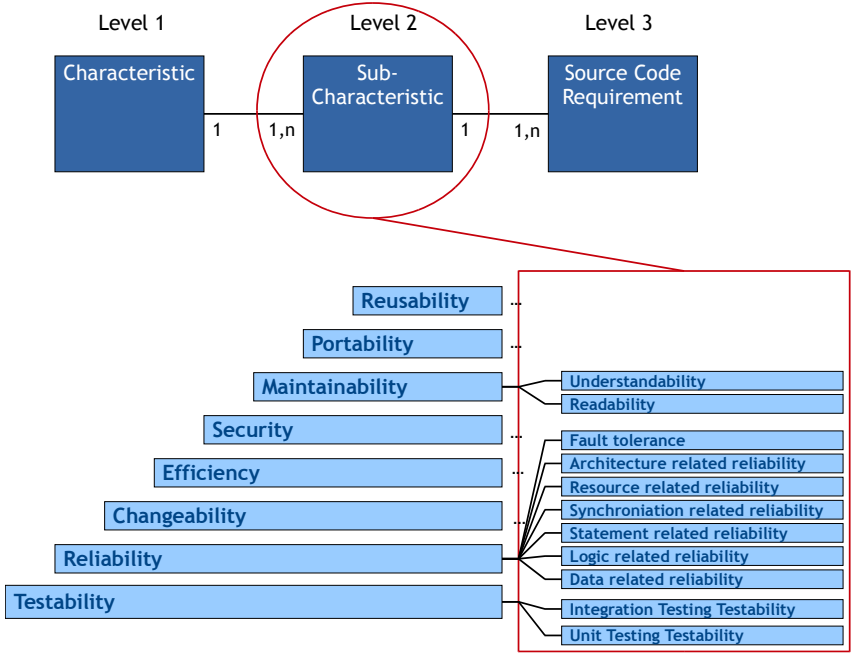
SQALE

- SQALE (Software Quality Assessment Based on Lifecycle Expectations) is a quality method to **evaluate technical debts** in software projects **based on the measurement of software characteristics**



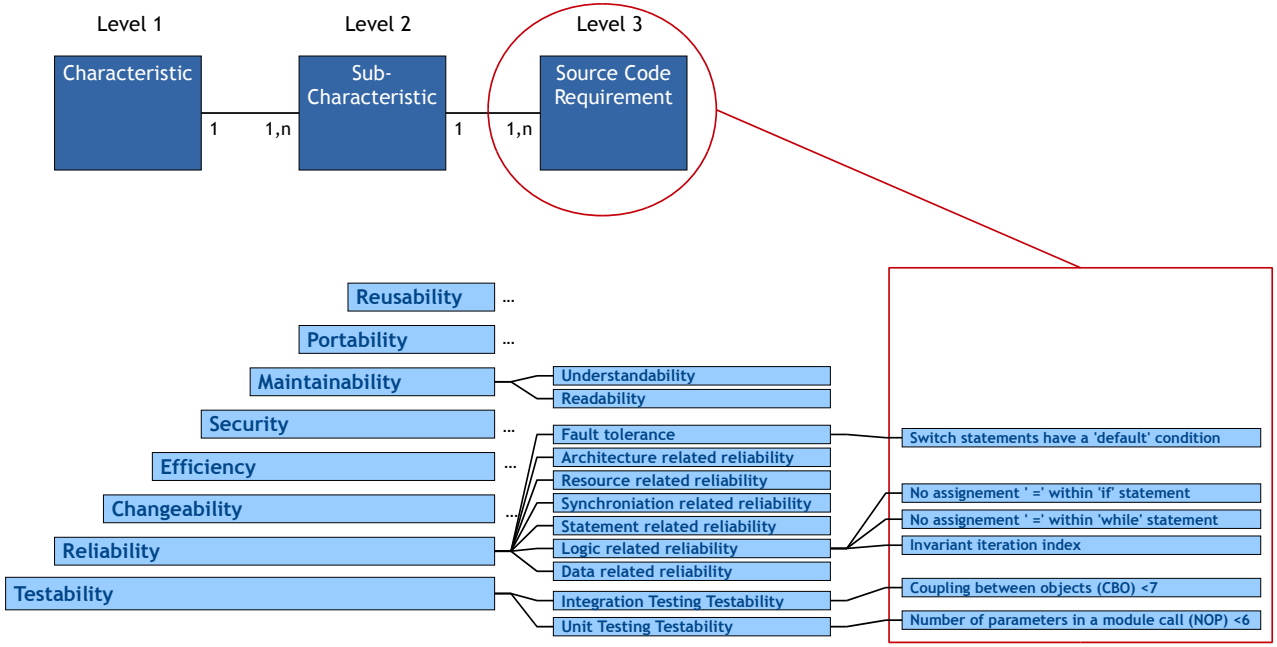
SQALE

- SQALE (Software Quality Assessment Based on Lifecycle Expectations) is a quality method to **evaluate technical debts** in software projects **based on the measurement of software characteristics**



SQALE

- SQALE (Software Quality Assessment Based on Lifecycle Expectations) is a quality method to **evaluate technical debts** in software projects **based on the measurement of software characteristics**



SQALE – (Non-)Remediation Function

- For each of the source code requirements we need to associate a **remediation function** that translates the non-compliances into remediation costs

NC Type Name	Description	Sample	Remediation Factor
Type1	Corrigible with an automated tool, no risk	Change in the indentation	0.01
Type2	Manual remediation, but no impact on compilation	Add some comments	0.1
Type3	Local impact, need only unit testing	Replace an instruction by another	1
Type4	Medium impact, need integration testing	Cut a big function in two	5
Type5	Large impact, need a complete validation	Change within the architecture	20

- Non-remediation functions represent the cost to keep a non-conformity so a negative impact from the business point of view

NC Type	Description	Sample	Non-Remediation Factor
Blocking	Will or may result in a bug	Division by zero	5 000
High	Wil have a high/direct impact on the maintainance cost	Copy and paste	250
Medium	Will have a medium/potential impact on the maintainance cost	Complex logic	50
Low	Wil have a low impact on the maintainance cost	Naming convention	15
Report	Very low impact, it is just a remediation cost report	Presentation issue	2

SQALE – Indexes and Rating

- Sums of all the remediation costs associated to a particular hierarchy of characteristics constitute an index:
  - SQALE Testability Index: STI
  - SQALE Reliability Index: SRI
  - SQALE Changeability Index: SCI
  - SQALE Efficiency Index: SEI
  - SQALE Security Index: SSI
  - SQALE Maintainability Index: SMI
  - SQALE Portability Index: SPI
  - SQALE Reusability Index: SRuI
  - **SQALE Quality Index: SQI (overall index)**

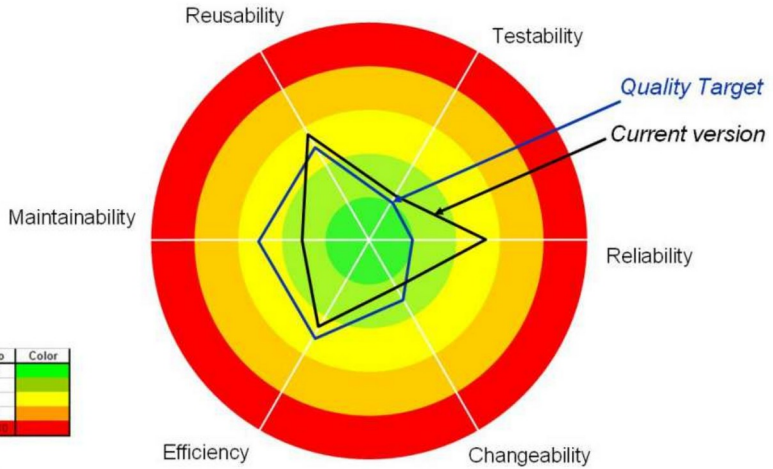
- Indexes can be used to build a rating value:

$$Rating = \frac{estimated\ remediation\ cost}{estimated\ development\ cost}$$

Rating	Up to	Color
A	1%	
B	2%	
C	4%	
D	8%	
E	∞	

- Example
  An artefact that has an estimated development cost of 300 hours and a STI of 8.30 hours, using the reference table on the left

$$Rating = \frac{8.30\ h}{300\ h} = 2.7\ \% \text{->} C$$



Rating	UpTo	Color
A	0.3	
B	1	
C	3	
D	10	
E	over 10	

Source: http://www.sqale.org/wp-content/uploads/2010/08/SQALE-Method-EN-V1-0.pdf

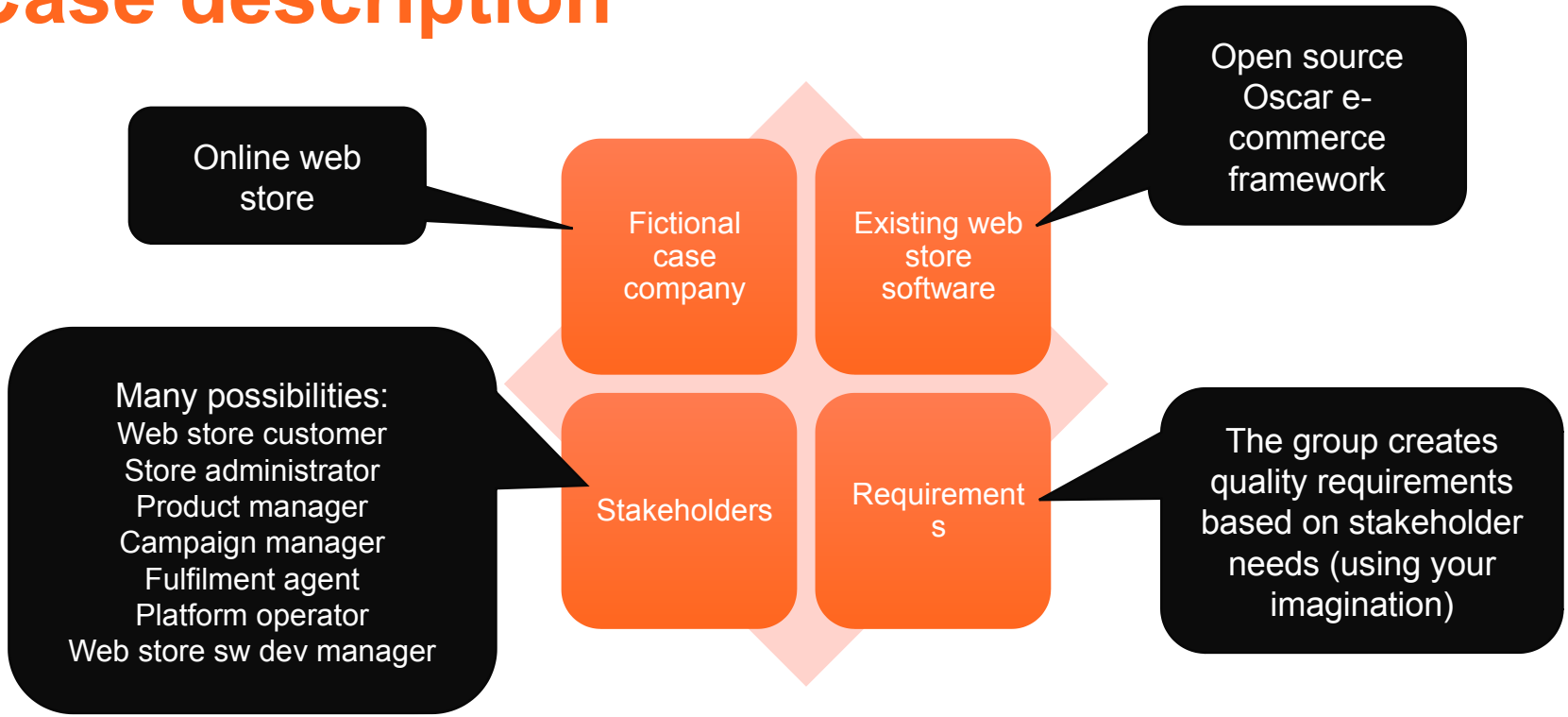
Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box, white-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: Black-box, white-box, manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6
45	5.11.2024	Guest lecture	Individual assignments 7, Group assignment 2
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3

Subject to changes

Group assignment: Final report

Aalto University
School of Science

Case description



Tasks and deliverables

Prepare	Create a quality model	Define quality control activities	Define a test plan	Execute a few tests	Make quality assessment
• Read the project case description • Choose two stakeholders to focus on • *Document as report introduction*	• Analyse stakeholder needs • Create a quality model according to ISO 25010 • *Document as a table + definitions*	• Tests • Inspections • Reviews • Metrics • Other appropriate quality control activities • *Document as table + text*	• At least 10 test cases (of which at least 1 automated unit test case, 1 automated acceptance test case, 1 manual test case) • *Document with inputs and expected outputs*	• Results from running the tests • You can reuse tests that you have already carried out in the course – but they must be properly documented • *Document test results*	• What is the current quality of the SUT for different stakeholders? • *Document as text*

Final deliverable: a single PDF. A report template is provided in the assignment

Next steps

Use the related reading for more details. Reserve enough time to set up your environment.

- Individual assignments in MyCourses
  - Static code analysis 2: Sonarqube
- Group assignments
  - Team testing

Deadline: before next lecture (Tuesday by 10:00)

- Software measurement

Deadline: before lecture in two weeks (12.11. by 10:00)

- Group assignments opening
  - Final report (DL: 26.11)

Getting help
Ask in the course chat, see Communication section in MyCourses
Personal questions by email

Study smarter, not harder!
- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ⏪